# 动物分类

1. 实验目的

数据集整理，训练一个神经网络，实现牛和羊的分类；

2. 实验环境

SUB KIT NX 嵌入式开发板。

3. 实验数据

实验采用牛羊数据集共 200 个样本，如图 4.1 所示。该数据集文件名进行了整理，每个图像都有名称和数字标签，名称和数字之间用“.”隔开。牛的数据集 100 张，命名为 cow.0.jpeg~cow.99.jpeg；羊的数据集 100 张，命名为 sheep.0.jpeg~sheep.99.jpeg。

图 4.1 牛羊数据集

4. 构造样本集

数据集已经按照类别进行分类，为了将数据读取到 pytorch 中 dataloader 类中，需要得到所有图像的地址，并按照一定的比例将数据集划分成训练集和测试集。

```
def read_split_data(root: str, val_rate: float = 0.2):   # root 表示图像存储路径，val_rate 表示测试集所占比率
    #以下找到所有类别（文件夹），并对应索引号，并通过 json 存储
        random.seed(0)   # 保证随机结果可复现，每次运行选取相同样本
        assert os.path.exists(root), "dataset root: {} does not exist.".format(root)
```

# 遍历文件夹，一个文件夹对应一个类别，获取文件夹名，all_class 为[“cow”,“sheep”]

all_class = [cla for cla in os.listdir(root) if os.path.isdir(os.path.join(root, cla))]

# 排序，保证顺序一致，保证文件夹顺序一致

all_class.sort()

# 生成类别名称以及对应的数字索引

class_indices = dict((k, v) for v, k in enumerate(all_class)) #v 是序号，k 是类别名

json_str = json.dumps(dict((val, key) for key, val in class_indices.items()), indent=4)#json 格式：“0”：“cow”，“1”:“sheep”

with open('class_indices.json', 'w') as json_file:

    json_file.write(json_str) #存储到硬盘

#以下整理图像的路径，并按照预设比例进行训练集和验证集划分，

train_images_path = []  # 存储训练集的所有图像路径，如‘sample\\cow\\cow.0.jpeg’,……

train_images_label = []  # 存储训练集图像对应标签信息，如：0,0,0……，与图像路径文件一一对应

val_images_path = []  # 存储验证集的所有图像路径

val_images_label = []  # 存储验证集图像对应索引信息

every_class_num = []  # 存储每个类别的样本总数

supported = [".jpg", ".JPG", ".png", ".PNG", ".jpeg"]  # 支持的文件后缀类型

# 遍历每个文件夹下的文件

for cla in all_class:

    cla_path = os.path.join(root, cla)

    # 遍历获取 supported 支持的所有文件路径

    images = [os.path.join(root, cla, i) for i in os.listdir(cla_path)

if os.path.splitext(i)[-1] in supported] # 例如：sample\\cow\\cow.0.jpeg,…，images 放 cow 或 sheep 路径下的所有图像具体路径

# 获取该类别对应的标签

image_class = class_indices[cla]

# 记录该类别的样本数量

every_class_num.append(len(images))

# 按比例随机采样验证样本

val_path = random.sample(images, k=int(len(images) * val_rate))#20%作为验证样本，val_path 是相应路径的列表

for img_path in images:

if img_path in val_path: # 如果该路径在采样的验证集样本中则存入验证集

val_images_path.append(img_path)

val_images_label.append(image_class)

else: # 否则存入训练集

**#自己补充**

print("{} images were found in the dataset.".format(sum(every_class_num)))

print("{} images for training.".format(len(train_images_path)))

print("{} images for validation.".format(len(val_images_path)))

return train_images_path, train_images_label, val_images_path, val_images_label

之后，继承 pytorch 中 dataset 类，创建自己的 dataset 类。

#以下具体到每一个图像

class MyDataSet(Dataset):

"""自定义数据集"""

```
    def __init__(self, images_path: list, images_class: list, transform=None):
        self.images_path = images_path
        self.images_class = images_class
        self.transform = transform

    def __len__(self):
        return len(self.images_path)#图像数目

    def __getitem__(self, item):
        img = Image.open(self.images_path[item]).convert('RGB')#确保三通道，根据模型输入要求
        # RGB 为彩色图像，L 为灰度图像
         label = self.images_class[item]

        #图像预处理
        if self.transform is not None:
            img = self.transform(img)

        return img, label

    @staticmethod
    def collate_fn(batch):
        #  官方实现的 default_collate 可以参考 https://github.com/pytorch/pytorch/blob/67b7e751e6b5931a9f45274653f4f653a4e6cdf6/torch/utils/data/_utils/collate.py
        images, labels = tuple(zip(*batch))
        images = torch.stack(images, dim=0)#图像分批为 Batch

        labels = torch.as_tensor(labels)
        return images, labels
```

在读取数据集并创建 dataloader 类时，首先需要对数据进行预处理。

```
S_Length = 128                                    # 样本尺寸
S_Width = 128
batch_size = 2
data_transform = transforms.Compose([transforms.Resize([S_Length, S_Width]),
                                     transforms.ToTensor()])
```

实例化自己的 dataset 类

```
train_dataset = MyDataSet(images_path=train_images_path,
                          images_class=train_images_label,
                          transform=data_transform)
```

并依据该 dataset 创建 dataloader

```
train_loader = torch.utils.data.DataLoader(train_dataset,
                                           batch_size=batch_size,
                                           shuffle=True,
                                           pin_memory=True,
                                           drop_last=True,
                                           collate_fn=train_dataset.collate_fn)
```

**#测试集自己补充**

5. 构建网络

所构建网络结构如图 4.3 所示。

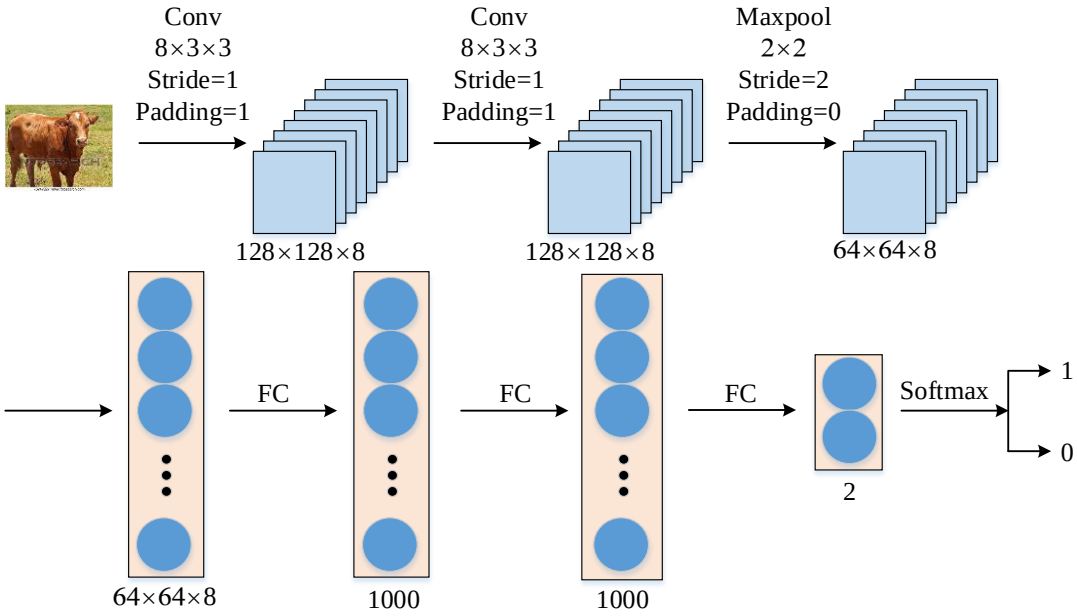


图 4.3 分类牛羊的网络结构

训练与测试

如果网络结构存储在一个独立的.py 文件中，在训练时需要将网络引入，引入方法如下：

```
from net_model import Net    #从 net_model.py（网络结构文件）导入网络模型
net = Net().cuda()           # 将网络传入 GPU
```

第一行代码类似于其他的 import 导入，Net 是已经创建的网络的类名称，其在 net_model.py 中已经自行构建，引入这个网络也就是引入这个类。程序中如果引用另一个文件中的类或函数，可以使用这种方法，要求引入的文件必须和当前运行的程序在同一文件夹下。第二行代码和其他类的实例化类似。网络训练主体代码如下：

```
def trainer(model, optimizer, data_loader, epoch):
    """
    定义训练脚本
    :param model: 训练模型
    :param optimizer: 优化器
    :param data_loader: 数据集
    :param epoch: 训练当前代数
```

```
    :return:
    """
    device = "cuda:0"

    model.train()

    accu_loss = torch.zeros(1).to(device)
    accu_num = torch.zeros(1).to(device)

    optimizer.zero_grad()

    sample_num = 0
    data_loader = tqdm(data_loader, file=sys.stdout)#可视化一代训练中的进度（进度条）
    for step, data in enumerate(data_loader):
        images, labels = data
        images, labels = images.to(device), labels.to(device)

        output = model(images)#图像传入网络
        sample_num += images.shape[0]
        criterion = nn.CrossEntropyLoss()                    # 定义损失函数
        loss = criterion(output, labels)

        pred_classes = torch.max(output, dim=1)[1]
        accu_num += torch.eq(pred_classes, labels).sum()
        loss.backward()
        accu_loss += loss.detach()
        data_loader.desc = "[train epoch {}] loss: {:.3f}, acc:{:.3f}".format(epoch,
```

```
accu_loss.item() / (step + 1),

accu_num.item() / (sample_num))

            if not torch.isfinite(loss):#loss 异常判断
                print('WARNING: non-finite loss, ending training ', loss)
                sys.exit(1)
            optimizer.step()
            optimizer.zero_grad()

        return accu_loss.item() / (step + 1), accu_num.item() / sample_num
```

之后循环调用该训练脚本

```
for epoch in range(epochs):
    train_loss, train_acc = trainer(net,
                                    optimizer,
                                    train_loader,
                                    epoch)
```

**#自己补充测试**

训练网络的过程中，可以打印输出中间结果，或利用 Matplotlib 进行可视化显示。网络训练结束后，可以将训练好的网络进行保存。

6. 实验任务

同学自己编写完整的构建样本集程序，测试样本集是否正常的程序，构建分类网络，训练网络程序，可视化训练过程，测试网络程序，模拟实际运用。